

Filière : Sciences de données, Big Data & IA

Mini Projet Java

Analyse des sentiments & Web Scrapping

Détection des Émotions dans les Commentaires Textuels sur Reddit

Réalisé par :

-
- DIDI Aya
 - EL KADRI Basma
 - AMAKASSOU Aziza
 - BAYCHOU Imane

Encadré par :

Prof. KARBIL Loubna

Année universitaire : 2025/2026

Résumé

Le but de ce projet était de scraper des commentaires en ligne (notamment depuis Reddit) et d'analyser automatiquement les sentiments exprimés à l'aide d'un modèle de machine learning qu'on a entraîné. L'objectif principal est de permettre une évaluation rapide et automatique de l'opinion publique sur un sujet donné, en classifiant les commentaires selon leur polarité (positif, négatif ou neutre).

Ce projet s'inscrit dans une démarche d'exploration de l'intelligence artificielle appliquée au traitement du langage naturel (NLP), en exploitant la puissance des réseaux neuronaux pour comprendre les émotions humaines à travers le texte. Il illustre une application concrète du machine learning au quotidien, notamment pour les entreprises souhaitant surveiller leur image de marque ou pour les chercheurs en sciences sociales analysant les tendances d'opinion.

Contrairement à d'autres systèmes nécessitant des ressources matérielles lourdes ou des corpus très spécifiques, notre solution repose sur des données accessibles et sur des outils open-source comme JavaFX pour l'interface utilisateur, et TensorFlow pour le traitement des données avec un modèle d'extension .h5. Cela rend notre outil accessible à toute personne disposant d'un ordinateur personnel, tout en assurant une efficacité de classification satisfaisante.

En définitive, ce projet démontre comment des techniques avancées d'intelligence artificielle peuvent être intégrées dans une application intuitive, pratique et adaptée à des besoins concrets, avec pour ambition de démocratiser l'analyse des sentiments dans le langage en ligne.

Table des matières

I.	Introduction	6
II.	Contexte du projet	7
A.	Présentation du besoin d'analyser les sentiments des commentaires sur Reddit.....	7
B.	Objectif principal du projet	7
III.	La Structure Globale du projet	8
A.	Diagramme de blocs du projet	8
1.	Data Processing (Traitement des données).....	8
2.	Training (Entraînement du modèle)	8
3.	Prédiction (Classification des textes)	8
B.	Diagramme de cas d'utilisation.....	9
1.	Acteurs Principaux	9
2.	Cas d'utilisation principaux	9
C.	Diagramme d'activité.....	10
Les étapes du diagramme :		11
1.	Start (Début).....	11
2.	Entrée de l'URL Reddit	11
3.	Validation de l'URL Reddit	11
4.	Appel à l'API Reddit	11
5.	Analyse de la réponse JSON	11
6.	Affichage des résultats	11
7.	Renvoi des émotions.....	11
8.	Afficher les statistiques	11
9.	End (Fin)	12
IV.	Architecture de l'Application et Communication entre les Composants.....	12
A.	Vue d'ensemble de l'architecture client-serveur	12
B.	Fonctionnement du Backend (Flask).....	12
1.	Serveur Flask.....	12
2.	Traitement des requêtes :.....	13
C.	Fonctionnement du Frontend (JavaFx)	13
1.	Structure de l'Interface Utilisateur (UI).....	13
2.	Interaction avec l'Utilisateur.....	13
D.	Communication entre le Frontend et le Backend	13
1.	Envoi des données du frontend vers le backend	14
2.	Extraction des commentaires :	14

3.	Analyse des émotions et réception des résultats	14
V.	Les résultats	14
VI.	Modèle d'Analyse des Émotions	14
A.	Présentation du Modèle	14
B.	Données Utilisées pour l'Entraînement	15
C.	Algorithmes Utilisés.....	15
1.	TF-IDF + Random Forest.....	15
2.	Réseau de Neurones (Deep Learning).....	15
D.	Développement et Entraînement du Modèle	15
1.	Chargement et préparation des données :	16
2.	Prétraitement des textes :	16
3.	Création et entraînement des modèles :	16
4.	Évaluation des performances :	16
5.	Sauvegarde des modèles :	16
6.	Déploiement et tests :	16
E.	Intégration dans l'Application	16
VII.	Démonstration fonctionnelle	17
A.	Les interfaces.....	17
1.	Page d'accueil :	17
2.	Page d'affichage des commentaires :	17
3.	Page d'analyse des émotions :	18
4.	Page de statistiques globales :	19
VIII.	Conclusion.....	21

Table des figures

Figure 1: Reddit logo.....	7
Figure 2: diagramme de blocs	8
Figure 3:Diagramme de cas d'utilisation	9
Figure 4:Diagramme d'activités	10
Figure 5:Flask logo.....	12
Figure 6: JavaFx logo	13
Figure 7:page accueil.....	17
Figure 8: affichage des commentaires.....	18
Figure 9: analyse des émotions des commentaires.....	18
Figure 10: graphe en barres 1	19
Figure 11: Graphique1 circulaire ; Statistiques des émotions détectés	20
Figure 12 : Graphique2 circulaire ; Statistiques des émotions détectés	20
Figure 13 : graphe en barres 2	21

I. Introduction

Dans un monde de plus en plus numérique, les interactions humaines passent souvent par le langage écrit : messages sur les réseaux sociaux, avis en ligne, e-mails, ou encore conversations via des chatbots. Détecter les émotions exprimées dans ces textes est devenu un enjeu central pour mieux comprendre les intentions, les besoins ou encore l'état émotionnel des utilisateurs. Ce domaine, appelé **détection des émotions**, constitue une extension de l'analyse de sentiments, en allant au-delà des simples polarités « positif / négatif / neutre » pour capturer des états émotionnels plus nuancés tels que la joie, la tristesse, la peur ou la colère.

Ce projet s'inscrit dans cette problématique. Il a pour **objectif principal de comparer deux approches de classification multi-étiquettes** pour la détection automatique des émotions dans des textes courts en anglais :

- Une approche **classique** basée sur une vectorisation TF-IDF suivie d'un classifieur Random Forest.
- Une approche **deep learning** s'appuyant sur un réseau de neurones entraîné sur les vecteurs TF-IDF.

Pour mener à bien cette étude, nous avons utilisé le **dataset GoEmotions**, publié par Google. Ce corpus contient plus de 58 000 exemples de commentaires issus de Reddit, chacun annoté avec une ou plusieurs émotions parmi 27 catégories (par exemple : amusement, colère, fierté, gratitude...) ainsi qu'une classe « neutre ». Il s'agit d'un dataset multi-étiquette, car un texte peut exprimer simultanément plusieurs émotions. Cette richesse en fait une référence dans le domaine de la détection émotionnelle en traitement automatique du langage (TALN)

II. Contexte du projet

A. Présentation du besoin d'analyser les sentiments des commentaires sur Reddit



Figure 1: Reddit logo

Reddit est une plateforme en ligne de discussion où les utilisateurs peuvent publier des messages (posts) et interagir entre eux à travers des commentaires. Ces commentaires sont souvent riches en opinions, émotions et sentiments, que ce soit pour exprimer de l'enthousiasme, de la frustration, de la joie, de la colère ou d'autres émotions humaines. L'énorme quantité de contenu générée par les utilisateurs sur Reddit présente une opportunité d'analyse pour extraire des insights à partir des émotions exprimées dans ces commentaires.

A cause du volume massif de commentaires sur Reddit, l'analyse manuelle devient pratiquement impossible. C'est là qu'interviennent les techniques d'analyse de texte automatisées, et en particulier l'analyse des sentiments à l'aide de modèles de Machine Learning (ML). Ces modèles permettent de traiter rapidement et efficacement de grandes quantités de données textuelles et d'en extraire des informations utiles sur les émotions qui y sont présentes.

B. Objectif principal du projet

L'objectif principal de ce projet est de développer une interface capable d'extraire les commentaires d'un post Reddit donné, puis d'analyser les émotions exprimées dans ces commentaires à l'aide d'un modèle de Machine Learning. Le projet est divisé en plusieurs parties complémentaires :

- **Extraction des commentaires** : À partir de l'URL d'un post Reddit, l'application doit extraire les commentaires associés via l'API Reddit. Cette étape consiste à récupérer les données brutes depuis le site web, qui sont ensuite traitées.
- **Analyse des émotions** : Une fois les commentaires extraits, l'application envoie ces textes au modèle de Machine Learning déployé sur un serveur Flask. Le modèle de ML analyse chaque commentaire et prédit les émotions qui y sont présentes (par exemple, joie, tristesse, colère, etc.).
- **Affichage des résultats** : Après l'analyse des émotions, l'interface présente les résultats sous une forme lisible pour l'utilisateur. Chaque commentaire est accompagné d'une évaluation de l'émotion principale qu'il véhicule, ainsi que d'autres informations pertinentes (comme le score de popularité, l'auteur, etc.).

III. La Structure Globale du projet

A. Diagramme de blocs du projet

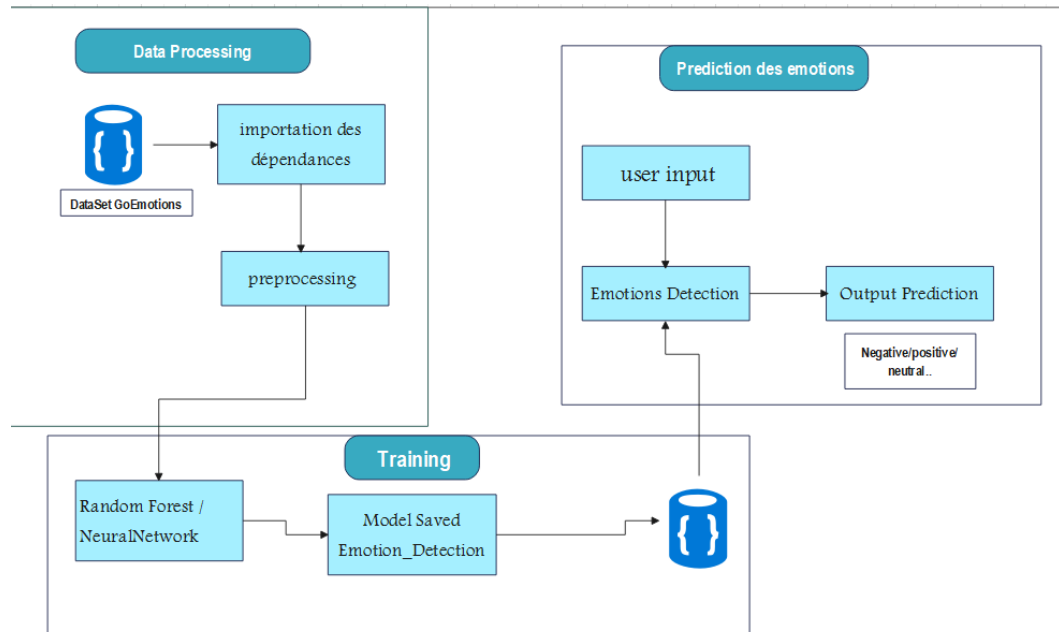


Figure 2: diagramme de blocs

Le projet est divisé en trois grandes parties principales :

1. Data Processing (Traitement des données)

Le dataset GoEmotions est utilisé comme base de données. Ce fichier CSV contient des phrases annotées avec différentes émotions. Ensuite, l'importation de toutes les bibliothèques nécessaires au traitement des données et à l'entraînement du modèle (pandas, sklearn, keras, nltk...). Puis, dans preprocessing, on traite les données (nettoyage, filtrage, suppression des valeurs manquantes) et prépare les textes pour l'entraînement du modèle.

2. Training (Entraînement du modèle)

Le modèle d'apprentissage supervisé utilisé est : soit un Random Forest, soit un Réseau de Neurones (Neural Network).

Après entraînement, le modèle est sauvegardé sous forme d'un fichier.

3. Prédiction (Classification des textes)

- Récupérer une saisie texte de l'utilisateur sous forme d'un lien sur Redit.
- Charger le modèle sauvegardé et prédire les émotions associées au texte.
- Les émotions détectées sont affichées en sortie pour l'utilisateur.

B. Diagramme de cas d'utilisation

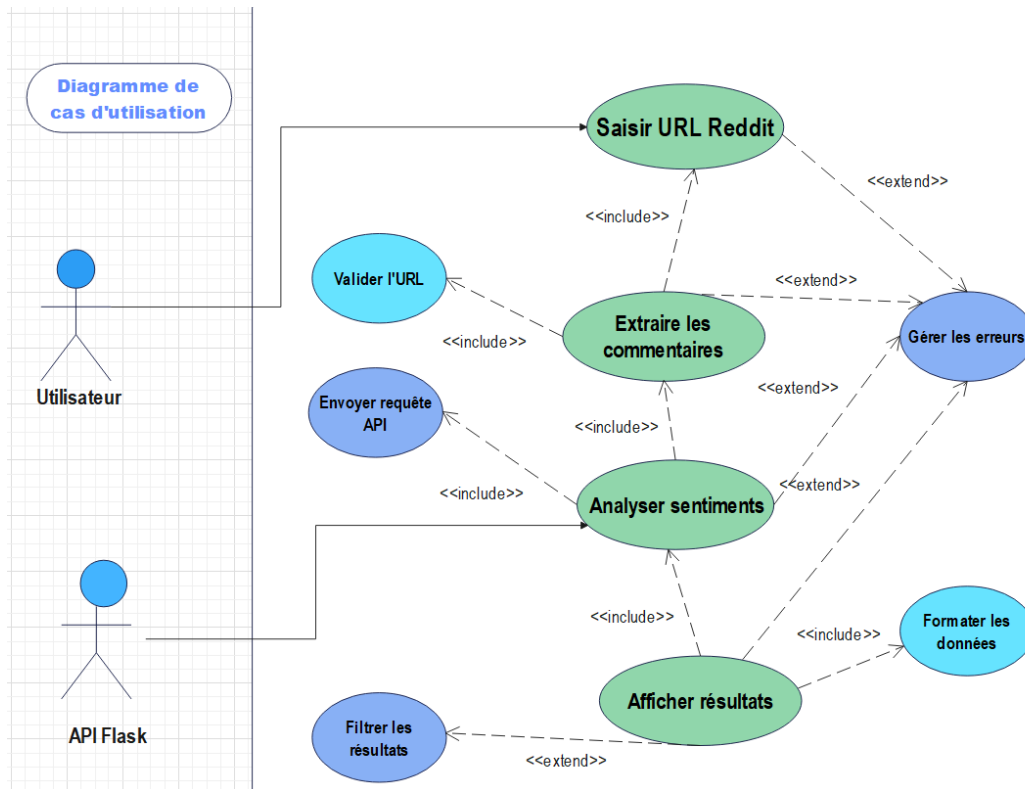


Figure 3: Diagramme de cas d'utilisation

Ce diagramme modélise les interactions entre les utilisateurs et le système pour :

- Extraire des commentaires Reddit
- Analyser leurs sentiments
- Afficher les résultats

1. Acteurs Principaux

L'**Utilisateur** représente la personne interagissant avec l'application via l'interface JavaFX, tandis que l'**API Flask** est considérée comme un acteur externe fournissant le service d'analyse de sentiments.

2. Cas d'utilisation principaux

Cinq cas d'utilisation principaux structurent le fonctionnement du système. Le processus commence par la saisie et validation d'une URL Reddit (**Saisir URL**). Vient ensuite l'extraction des commentaires (**Extraire commentaires**) qui s'appuie sur l'API Reddit. Le cœur métier du système réside dans l'analyse des sentiments (**Analyser sentiments**) qui fait appel à l'API Flask. Enfin, les résultats sont présentés à l'utilisateur (**Afficher résultats**) sous forme d'interface graphique.

Les relations entre ces cas d'utilisation suivent des patterns précis. Certaines étapes comme la validation de l'URL sont obligatoires (relation d'inclusion <<include>>), tandis que d'autres fonctionnalités comme le filtrage des résultats sont optionnelles (relation d'extension <<extend>>).

C. Diagramme d'activité

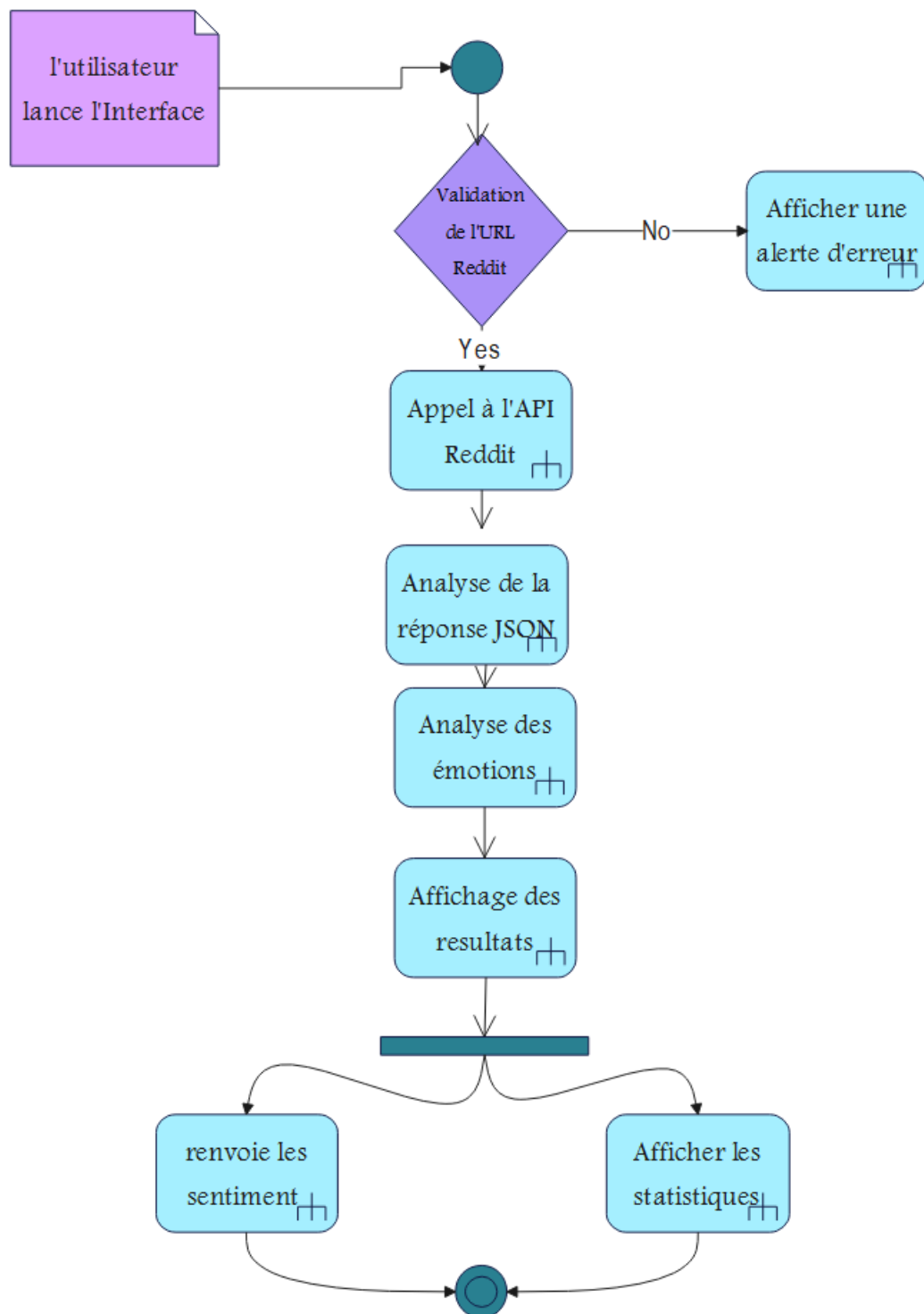


Figure 4:Diagramme d'activités

➤ Les étapes du diagramme :

1. Start (Début)

Le processus commence lorsque l'utilisateur lance l'interface. Ce nœud marque le début de l'activité de l'extraction des commentaires et de l'analyse des émotions.

2. Entrée de l'URL Reddit

L'utilisateur entre l'URL d'un post Reddit dans l'interface utilisateur de l'application (via un champ de texte). Cette étape est essentielle pour qu'il puisse cibler le post spécifique à partir duquel les commentaires seront extraits.

3. Validation de l'URL Reddit

Après l'entrée de l'URL, l'application valide si l'URL fournie correspond à une structure Reddit valide (par exemple, un lien qui mène à un post de commentaire). Si l'URL est incorrecte, une alerte d'erreur est affichée.

4. Appel à l'API Reddit

Si l'URL est valide, l'application fait un appel HTTP à l'API de Reddit pour récupérer les commentaires associés au post. Cette étape envoie une requête GET pour obtenir la réponse JSON contenant les commentaires.

5. Analyse de la réponse JSON

Une fois la réponse obtenue, le système analyse le fichier JSON reçu de Reddit pour en extraire les informations pertinentes (auteur, score, corps du commentaire, etc.).

6. Affichage des résultats

Après l'analyse réussie des commentaires, ces derniers sont présentés à l'utilisateur dans l'interface graphique en utilisant des composants JavaFX comme VBox et Label. Chaque commentaire est affiché sous forme de "carte" contenant l'auteur, le score, et le texte du commentaire.

7. Renvoi des émotions

À cette étape, l'application envoie chaque commentaire au modèle d'analyse des émotions via une API Python (ou un autre service de traitement) qui renvoie le sentiment (positif, négatif, neutre) du commentaire. Cette analyse permet de comprendre l'émotion dominante dans les commentaires Reddit.

8. Afficher les statistiques

L'interface permet également à l'utilisateur de visualiser des statistiques détaillées sur les commentaires extraits.

9. End (Fin)

Une fois toutes les étapes du processus exécutées (avec ou sans erreur), l'activité se termine, soit après l'affichage des commentaires et l'analyse des émotions, soit après l'affichage d'une erreur. Ce nœud marque la fin de l'activité dans le diagramme.

IV. Architecture de l'Application et Communication entre les Composants

A. Vue d'ensemble de l'architecture client-serveur

Le projet utilise une architecture client-serveur où le **frontend** est développé en **JavaFX**, et le **backend** est implémenté avec Flask (un micro-framework Python). Le frontend JavaFX est responsable de l'interaction avec l'utilisateur, tandis que le backend Flask sert de moteur pour l'analyse des émotions des commentaires Reddit.

Cette séparation en deux parties distinctes (frontend et backend) permet de bénéficier des avantages des deux technologies tout en rendant l'application plus modulaire et facile à maintenir. Le frontend gère l'interface graphique et l'interaction utilisateur, tandis que le backend se charge des calculs intensifs (comme l'analyse des émotions) et des interactions avec l'API Reddit.

B. Fonctionnement du Backend (Flask)

Le backend Flask est responsable du traitement des demandes d'analyse des émotions envoyées par le frontend :

1. Serveur Flask



Figure 5: Flask logo

Le serveur Flask expose une API HTTP qui attend des requêtes POST contenant des commentaires extraits de Reddit. Il utilise le modèle de Machine Learning qu'on a entraînée, qui est stocké sous le fichier **AnalyseEmotion_model.h5**, pour analyser les émotions exprimées dans ces commentaires.

2. Traitement des requêtes :

Lorsqu'une requête POST contenant un commentaire est reçue,

1. Flask pré-traite le texte.
2. Le modèle renvoie une prédiction sous forme de réponse JSON indiquant l'émotion du texte (par exemple, joie, tristesse, colère).
3. Flask renvoie cette prédiction au frontend.

C. Fonctionnement du Frontend (JavaFx)



Figure 6: JavaFx logo

Le frontend de l'application est développé en JavaFX, un framework Java moderne permettant de créer des interfaces graphiques riches et interactives. Dans ce projet, JavaFX joue un rôle essentiel en assurant l'interaction avec l'utilisateur, la présentation des données extraites de Reddit et l'affichage des résultats d'analyse des émotions.

1. Structure de l'Interface Utilisateur (UI)

L'interface utilisateur est construite autour d'une architecture basée sur le Scene Graph de JavaFX. Ce modèle hiérarchique organise les éléments visuels (nœuds) tels que les boutons, les champs de texte, les labels et les conteneurs ('VBox', 'StackPane') pour créer une interface cohérente et réactive. Chaque élément de l'UI est représenté par un nœud dans le graph, permettant une gestion efficace des événements et des interactions utilisateur.

2. Interaction avec l'Utilisateur

L'utilisateur commence par entrer l'URL d'un post Reddit dans un champ de texte. Lorsqu'il soumet l'URL, l'application valide sa structure et, si elle est correcte, envoie une requête HTTP au backend Flask pour récupérer les commentaires associés. Cette interaction est gérée de manière asynchrone pour éviter de bloquer l'interface utilisateur pendant le traitement des données.

D. Communication entre le Frontend et le Backend

La communication entre le frontend et le backend se fait via des requêtes HTTP simples :

1. Envoi des données du frontend vers le backend

- L'utilisateur entre un lien Reddit dans le champ de texte du frontend.
- L'application frontend envoie l'URL du post Reddit au backend, où un processus d'extraction des commentaires est lancé.

2. Extraction des commentaires :

L'API Reddit est appelée pour extraire les commentaires associés à l'URL du post Reddit. Une fois que les commentaires sont récupérés, le frontend envoie chaque commentaire à l'API Flask pour l'analyse des émotions.

3. Analyse des émotions et réception des résultats

Le modèle d'apprentissage automatique pré-entraîné dans Flask analyse les émotions dans les commentaires et renvoie la prédiction sous forme de JSON au frontend.

V. Les résultats

a) Les commentaires

Les commentaires seront extraits, ainsi que les émotions associées à chaque commentaire. Les commentaires sont présentés sous forme de cartes contenant le texte du commentaire, le nom de l'auteur et la prédiction d'émotion (par exemple, joie, colère, tristesse, etc.).

b) Statistiques et Visualisation

L'application offre également une fonctionnalité de visualisation des statistiques, permettant à l'utilisateur de voir la répartition des émotions (positif, négatif, neutre) parmi les commentaires extraits. Ces statistiques sont présentées sous forme de graphiques et de chiffres clés, offrant ainsi une analyse approfondie des sentiments exprimés.

VI. Modèle d'Analyse des Émotions

A. Présentation du Modèle

Dans ce projet, nous avons développé un modèle capable de prédire l'émotion associée à un commentaire textuel extrait d'une publication Reddit. L'objectif principal est d'analyser automatiquement les sentiments exprimés dans les commentaires utilisateurs pour en tirer des informations utiles, telles que la tonalité générale ou l'état émotionnel du public.

B. Données Utilisées pour l'Entraînement

Pour entraîner notre modèle d'analyse des émotions, nous avons utilisé le dataset **GoEmotions**, où chaque phrase est associée à une ou plusieurs étiquettes émotionnelles. Ce dataset couvre plusieurs classes d'émotions comme la joie, la tristesse, la colère, la peur, la surprise, etc.

Avant l'entraînement, les données ont été prétraitées pour améliorer la qualité de l'apprentissage, incluant des étapes telles que :

- La suppression des caractères spéciaux,
- La conversion en minuscules,
- La tokenisation, et le filtrage des mots inutiles (stop words).

C. Algorithmes Utilisés

Deux **principales approches** ont été adoptées pour construire le modèle :

1. TF-IDF + Random Forest

Les commentaires ont d'abord été vectorisés en utilisant la méthode TF-IDF (Term Frequency - Inverse Document Frequency), qui transforme les textes en vecteurs numériques pondérant l'importance des mots. Ensuite, un classifieur Random Forest a été utilisé pour la prédiction des émotions.

2. Réseau de Neurones (Deep Learning)

Dans une seconde approche, nous avons construit un modèle de réseau de neurones à l'aide de Keras. Le texte a été converti en séquences numériques et passé à travers des couches denses pour capturer les patterns complexes liés aux émotions.

➤ Chaque approche présente des avantages :

- Random Forest est rapide à entraîner et interpréter.
- Les réseaux de neurones offrent de meilleures performances sur de grands ensembles de données, au prix d'un temps de calcul plus élevé.

D. Développement et Entraînement du Modèle

Le développement du modèle d'analyse des émotions a été réalisé en Python, en utilisant plusieurs bibliothèques spécialisées :

- **Pandas** pour la manipulation des données.
- **NLTK** pour le prétraitement des textes (suppression du bruit, élimination des stopwords et lemmatisation).
- **Scikit-learn** pour la vectorisation TF-IDF, la création d'un pipeline et l'entraînement d'un modèle Random Forest.
- **TensorFlow/Keras** pour la création et l'entraînement d'un réseau de neurones profond.
- **TQDM** pour le suivi de la progression lors du prétraitement.

Le processus de développement a suivi les étapes principales suivantes :

1. Chargement et préparation des données

Le dataset GoEmotions a été chargé. Les exemples jugés très ambigus ont été filtrés, et seules les colonnes pertinentes représentant les émotions ont été conservées.

2. Prétraitement des textes

Les textes ont été normalisés en minuscules, nettoyés de toute ponctuation ou caractère non alphabétique, débarrassés des mots vides (stopwords) et lemmatisés pour uniformiser les formes des mots.

3. Création et entraînement des modèles

- **Approche classique** : Un pipeline combinant une vectorisation TF-IDF et un modèle Random Forest (multi-étiquette) a été construit et entraîné.
- **Approche Deep Learning** : Un réseau de neurones séquentiel composé de plusieurs couches denses et de couches de dropout a été développé. Il prend en entrée les représentations TF-IDF pour prédire les différentes émotions.

4. Évaluation des performances

Les deux modèles ont été évalués sur un jeu de test à l'aide de métriques adaptées aux problèmes multi-étiquettes, telles que la précision, le rappel, le F1-score et l'accuracy par émotion.

5. Sauvegarde des modèles

Le réseau de neurones entraîné a été sauvegardé au format .h5 et le vectoriseur TF-IDF sous forme de fichier .pkl, afin de permettre une utilisation rapide pour des prédictions en temps réel.

6. Déploiement et tests

Quelques exemples de phrases ont été utilisés pour démontrer les performances de prédiction des deux modèles (Random Forest et Réseau de Neurones).

E. Intégration dans l'Application

Pour intégrer notre modèle d'analyse émotionnelle dans l'application Java, nous avons utilisé une API web développée avec Flask.

Le modèle est exposé via une route /predict, permettant à l'application Java d'envoyer un commentaire en POST et de recevoir l'émotion prédite sous forme de réponse JSON.

Ainsi, l'application Java communique avec le modèle sans avoir à intégrer directement Python dans son code. Cela garantit modularité, performance et facilité de maintenance.

VII. Démonstration fonctionnelle

A. Les interfaces

1. Page d'accueil

- La page d'accueil permet à l'utilisateur de saisir le lien d'un post Reddit afin d'extraire les commentaires associés.

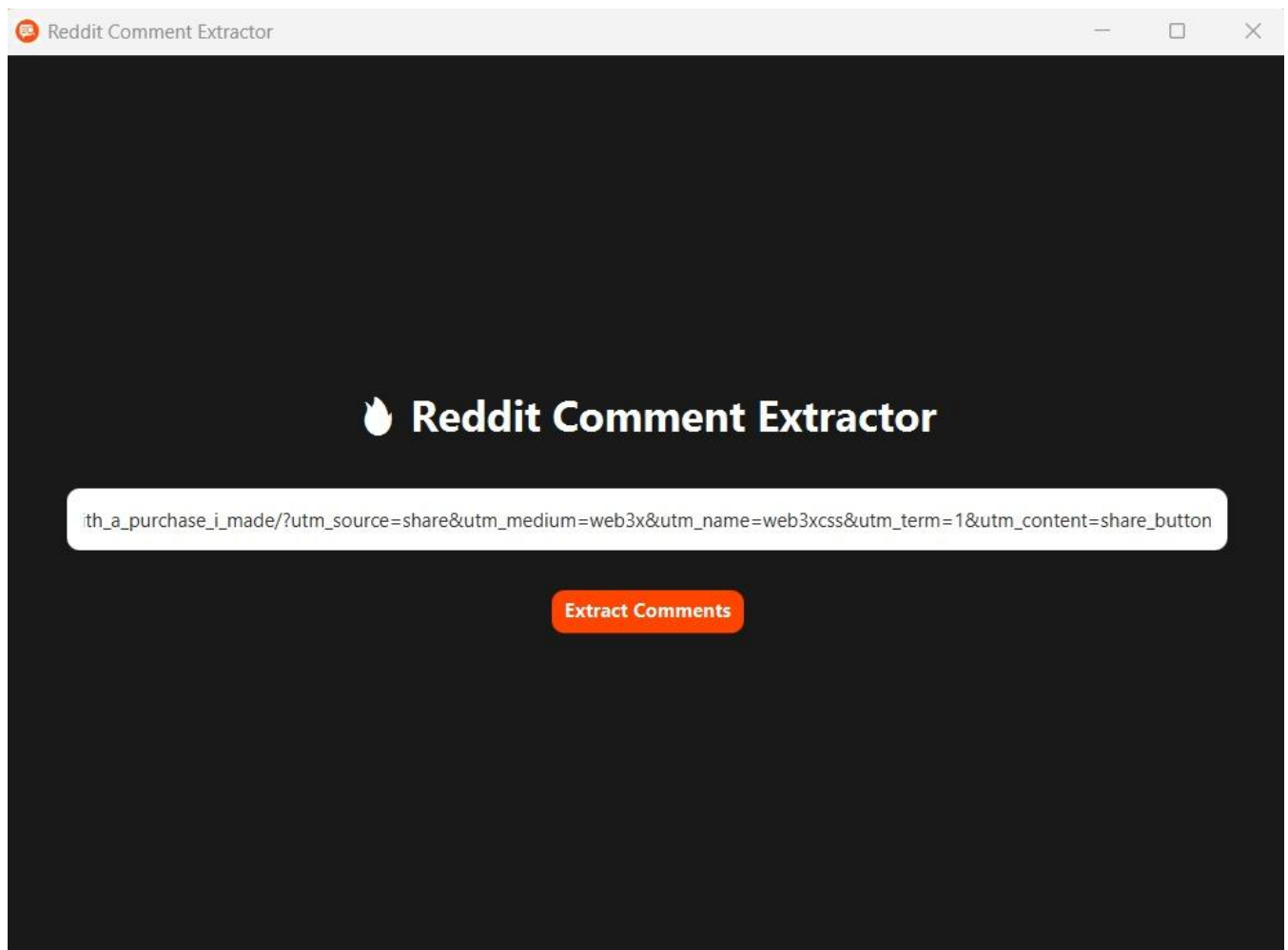


Figure 7:page accueil

2. Page d'affichage des commentaires

Cette page affiche les commentaires extraits ainsi que leurs informations, telles que l'utilisateur et le nombre de likes. Après consultation, l'utilisateur peut revenir à la page d'accueil en cliquant sur le bouton "Retour à l'accueil".

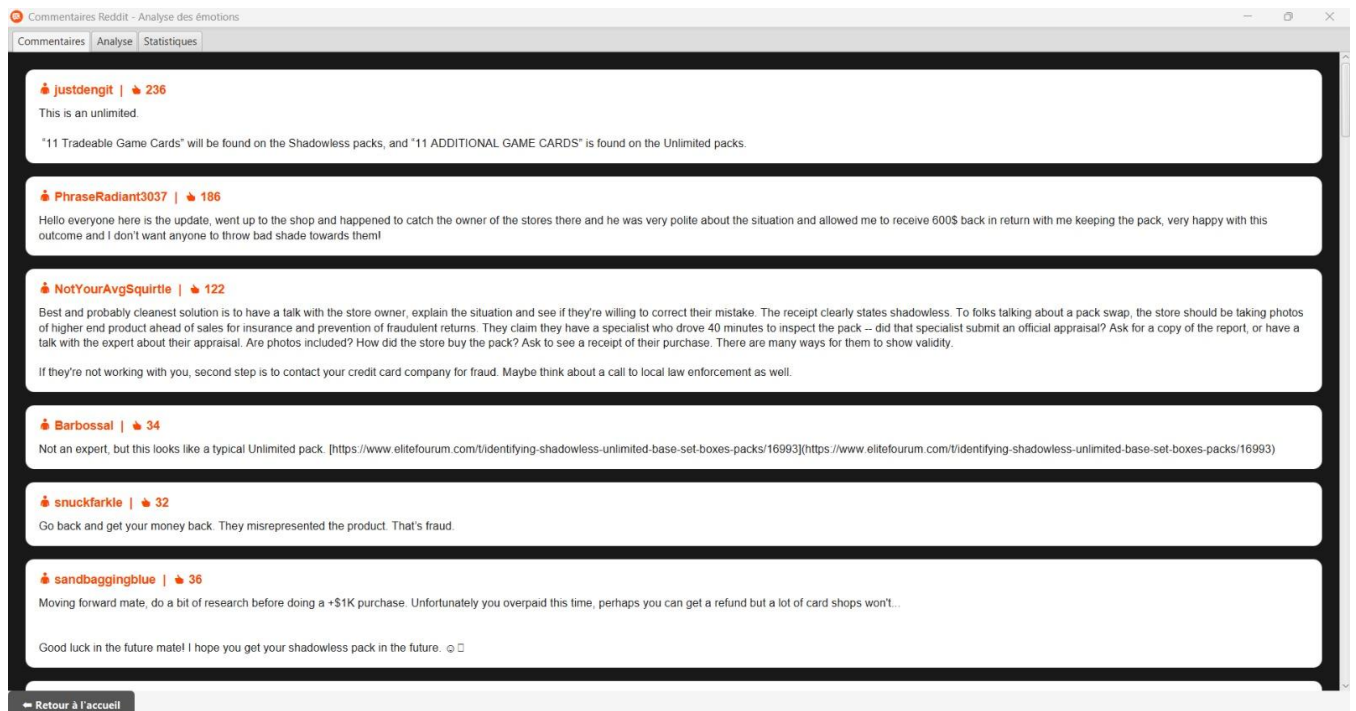


Figure 8: affichage des commentaires

3. Page d'analyse des émotions :

Cette interface affiche l'émotion détectée pour chaque commentaire, accompagnée de son pourcentage de confiance.

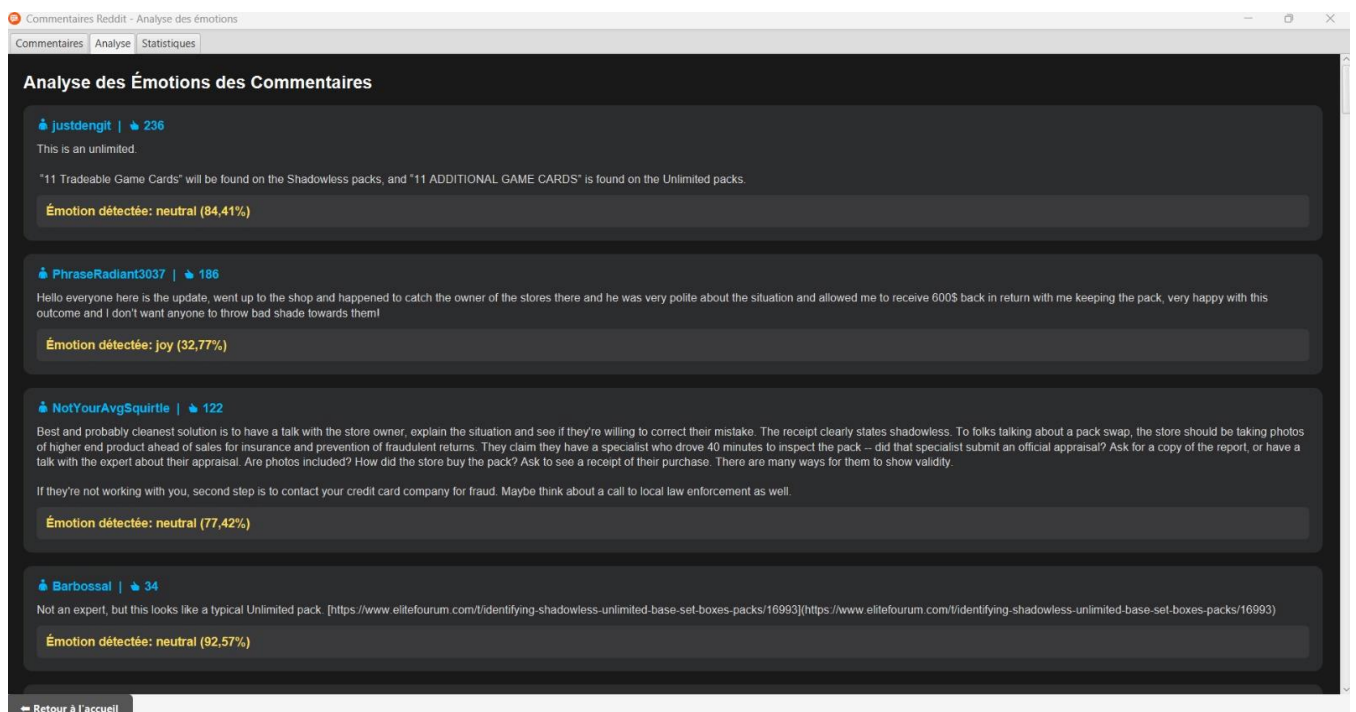


Figure 9: analyse des émotions des commentaires

4. Page de statistiques globales :

Cette page présente une vue d'ensemble des statistiques globales des commentaires extraits. Elle contient un graphique en barres qui illustre la répartition des différentes émotions détectées, ainsi qu'un graphique circulaire (camembert) qui montre la proportion de chaque émotion par rapport au total. Ces visualisations permettent à l'utilisateur d'avoir une analyse rapide et intuitive des données.

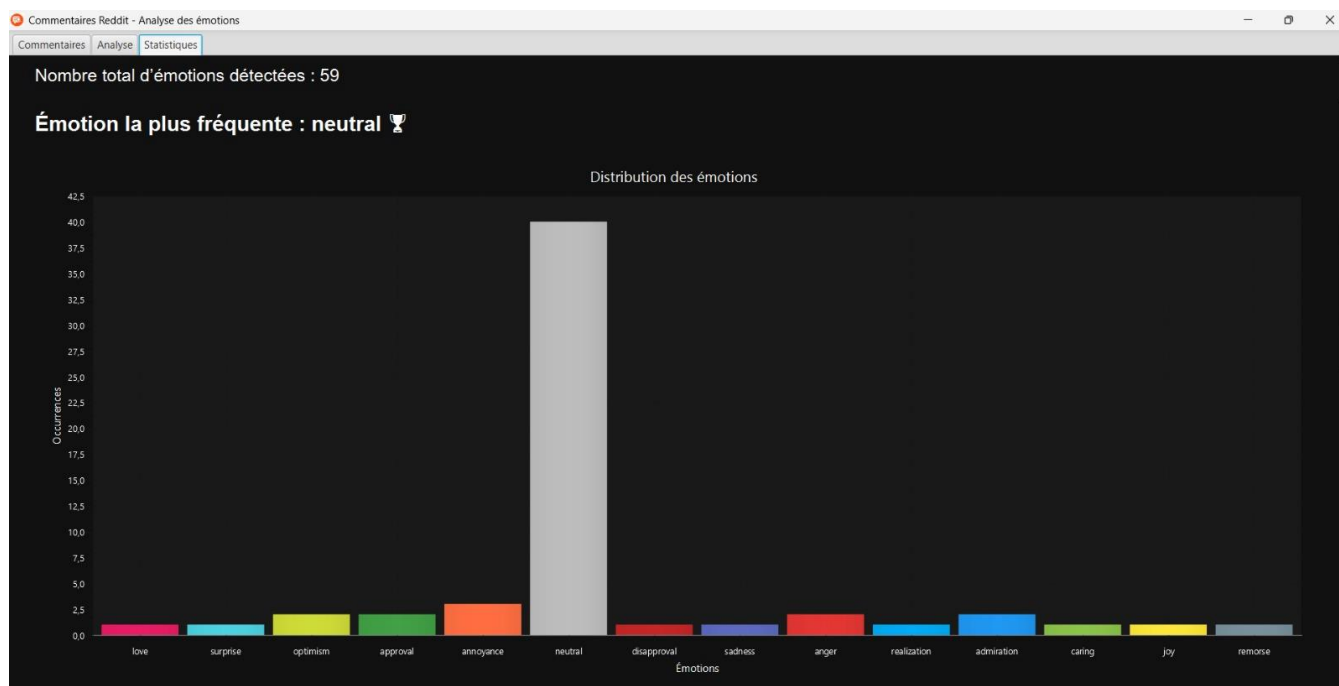


Figure 10: graphe en barres 1

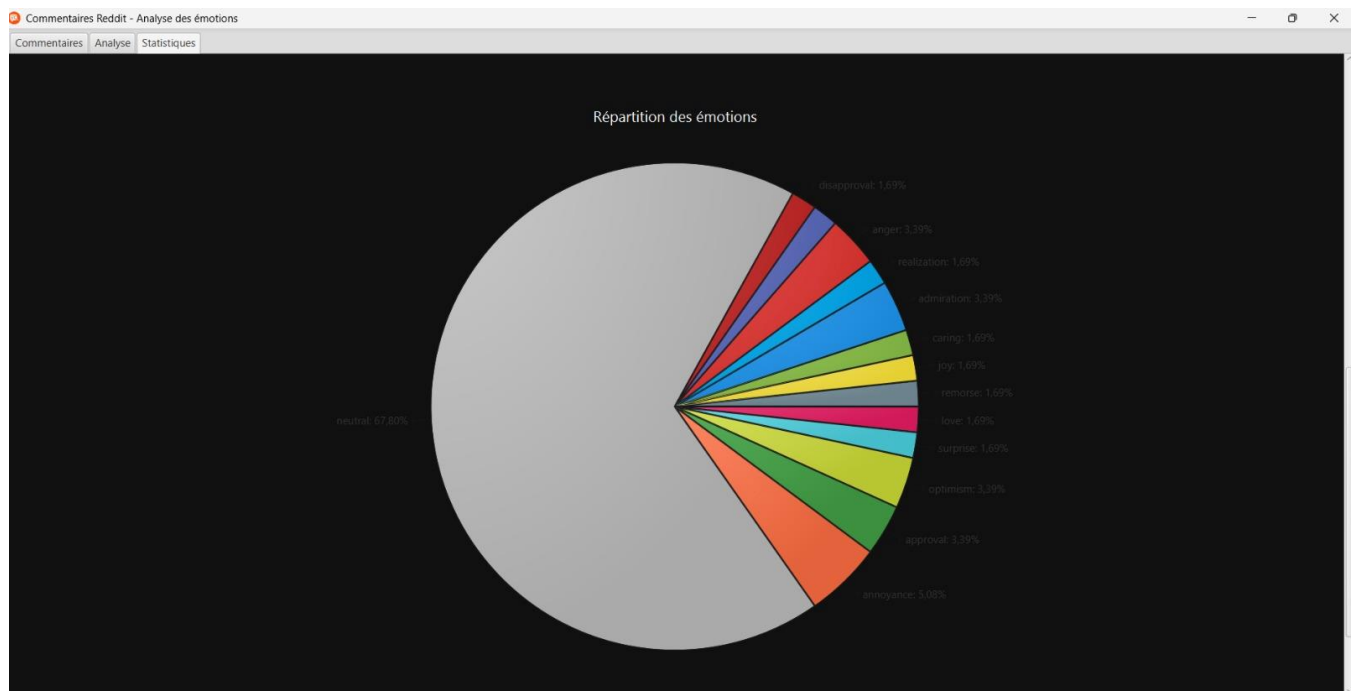


Figure 11: Graphique1 circulaire ; Statistiques des émotions détectés

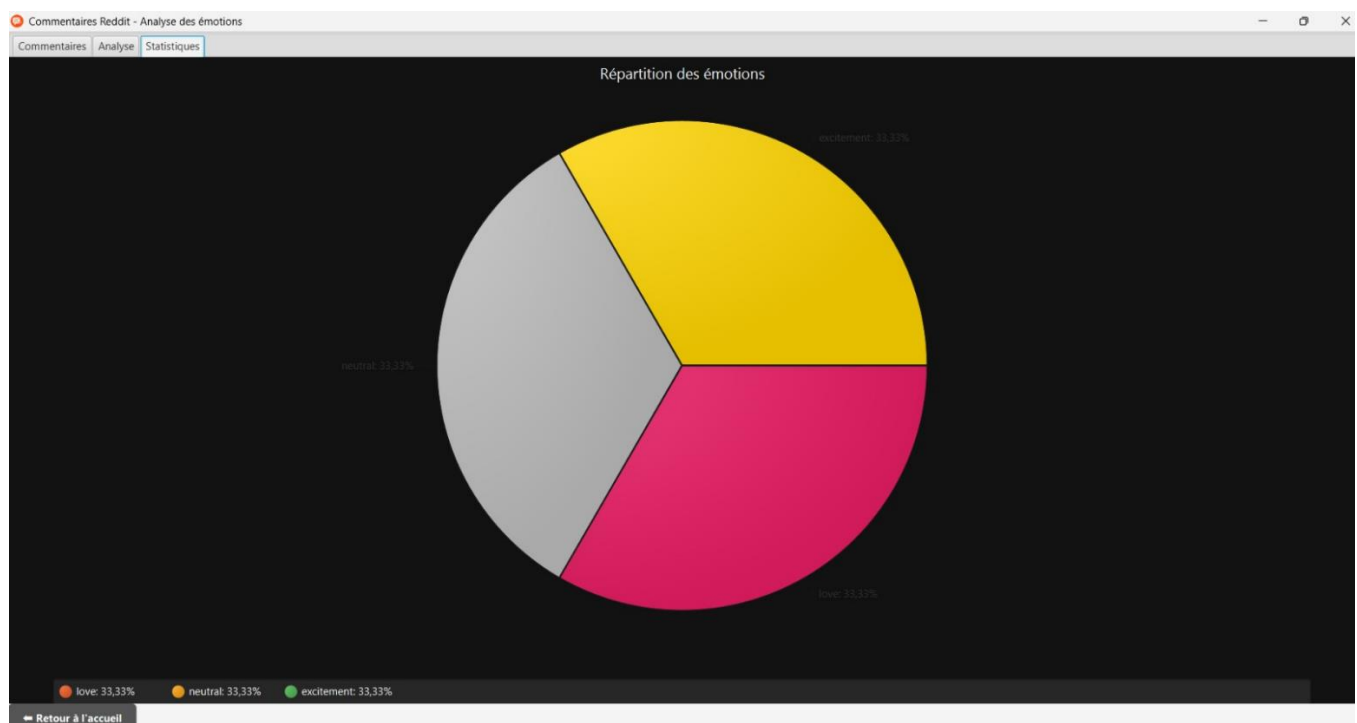
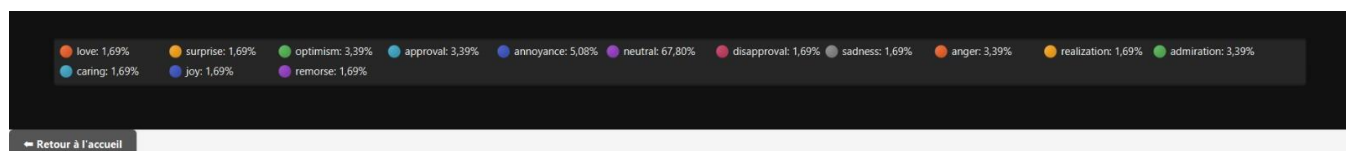


Figure 12 : Graphique2 circulaire ; Statistiques des émotions détectés

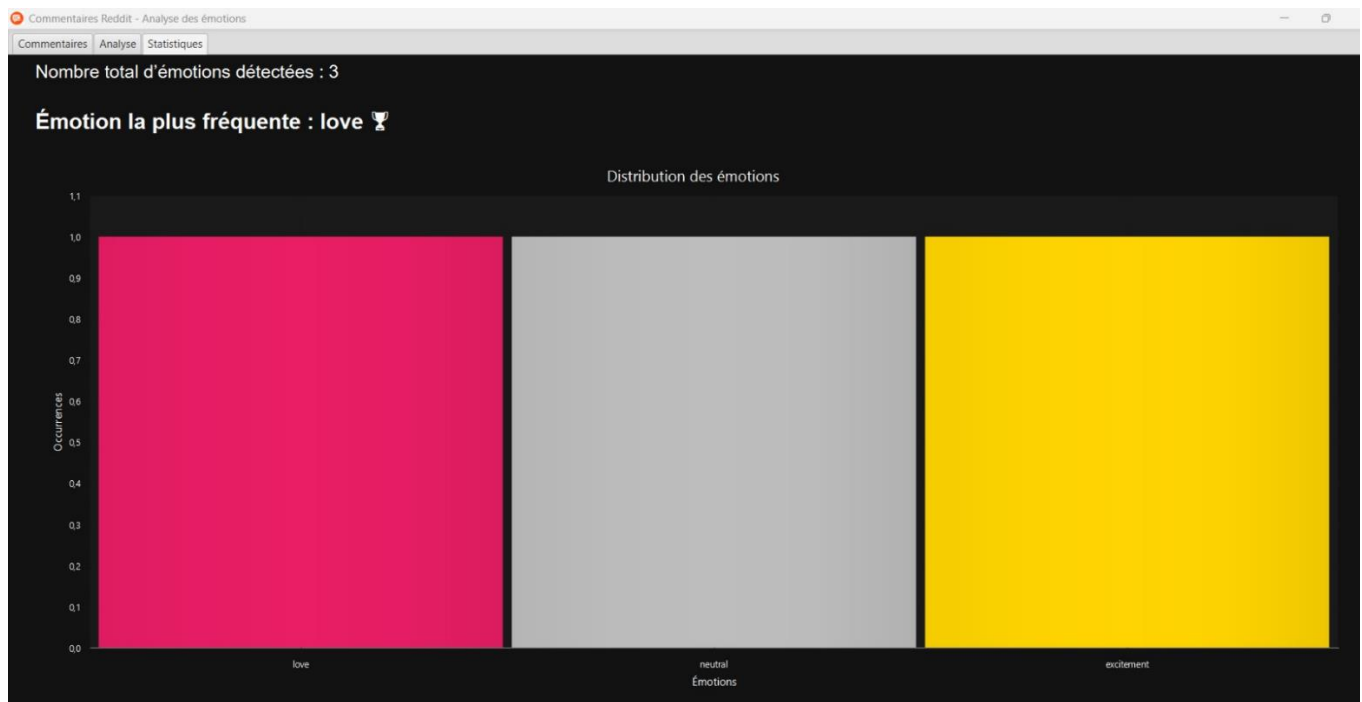


Figure 13 : graphe en barres 2

VIII. Conclusion

Ce projet a permis de développer une solution innovante pour l'analyse automatisée des émotions dans les commentaires Reddit, en combinant l'extraction de données via l'API Reddit et l'analyse des sentiments grâce à un modèle de Machine Learning. Grâce à cette approche, nous avons pu transformer un volume massif de données textuelles en informations structurées et exploitables, offrant ainsi des insights précieux sur les émotions exprimées par les utilisateurs.

L'interface développée facilite non seulement la récupération des commentaires, mais aussi leur analyse en temps réel, ce qui permet une compréhension rapide des tendances émotionnelles au sein d'une discussion. Les résultats obtenus démontrent l'efficacité des techniques de traitement automatique du langage naturel (NLP) pour interpréter les sentiments à grande échelle, ouvrant la voie à des applications variées, telles que l'analyse de l'opinion publique, le suivi de la satisfaction des utilisateurs ou encore la détection précoce de contenus problématiques.

En conclusion, ce projet illustre comment le Machine Learning et l'analyse de données peuvent être utilisés pour extraire du sens à partir de contenus générés par les utilisateurs, tout en offrant une base solide pour des améliorations futures, comme l'intégration de modèles plus avancés ou l'élargissement à d'autres plateformes sociales.

Ce travail confirme l'importance de l'automatisation dans l'analyse des sentiments et ouvre des perspectives prometteuses pour des recherches futures en NLP et en sciences sociales computationnelles.